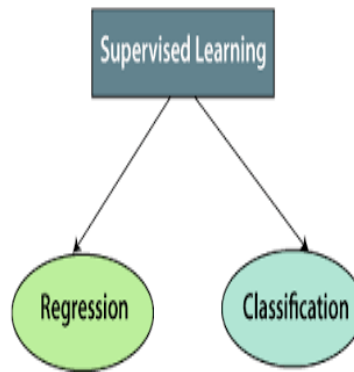


Data splitting

Introduction:

Machine learning algorithms are classified into supervised, unsupervised, semi-supervised and reinforced algorithms. Supervised algorithm is those that generally use labeled dataset to predict the output variable. Further the supervised classifier is divided into **regression and classification algorithms**.



- **Classification:** A classification problem is when the output variable is a category, such as “Red” or “blue”, “disease” or “no disease”.
- **Regression:** A regression problem is when the output variable is a real value, such as “dollars” or “weight”

Supervised machine learning is about creating models that precisely map the given inputs (independent variables, or **predictors**) to the given outputs (dependent variables, or **responses**).

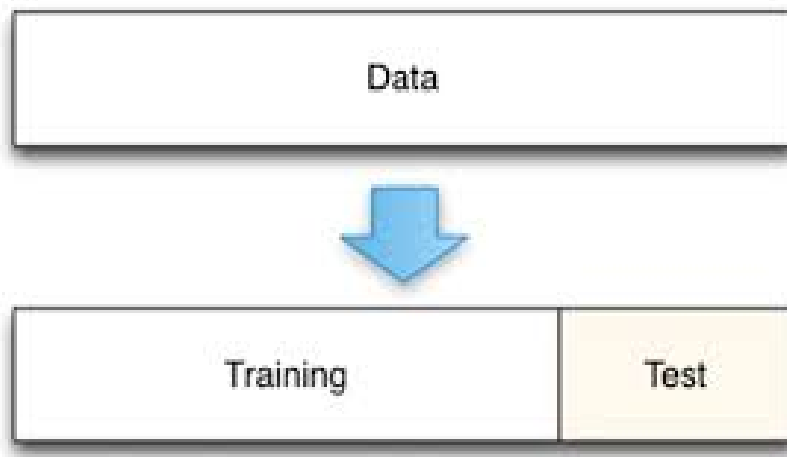
What is predictive modelling?

Predictive modeling is a mathematical process used to predict future events or outcomes by analyzing patterns in a given set of input data.

Training, Validation, and Test Sets

Splitting of dataset is essential for an unbiased evaluation of prediction performance and also avoids overfitting of data. In most cases, it is enough to split the dataset randomly into three subsets:

1. **The training set** is applied to train, or **fit**, your machine learning model. For example, we use the training set to find the optimal weights, or coefficients, for linear regression, logistic regression, or neural networks.
2. **The validation set** is used for unbiased model evaluation during hyperparameter tuning. For example, when you want to find the optimal number of neurons in a neural network or the best kernel for a support vector machine, you experiment with different values. For each considered setting of hyperparameters, you fit the model with the training set and assess its performance with the validation set.
3. **The test set** is needed for an unbiased evaluation of the final model. Test data should not be used for fitting or validation



The entire dataset must be split into training data and test data. *The `train_test_split()` function is used to split the data it is imported from `sklearn.model_selection import train_test_split`.*

Underfitting and Overfitting of Dataset

Splitting a dataset might also be important for detecting if your model suffers from one of two very common problems, called underfitting and overfitting:

1. Underfitting is usually the consequence of a model being unable to encapsulate the relations among data. For example, this can happen when trying to represent nonlinear relations with a linear model. Underfitted models will likely have poor performance with both training and test sets.
2. Overfitting usually takes place when a model has an excessively complex structure and learns both the existing relations among data and noise. Such models often have bad generalization capabilities. Although they work well with training data, they usually yield poor performance with unseen (test) data.

split training and testing data sets in Python using `train_test_split()` of sci-kit learn.

Python provides `train_test_split` from `sklearn` package to split the dataset into training and testing data.

Syntax: **`import numpy as np`**
`from sklearn.model_selection import train_test_split`

The `train_test_split` function is provided with following options. **options** are the optional keyword arguments that you can use to get desired behavior:

- **train_size** is the number that defines the size of the training set. If you provide a float, then it must be between 0.0 and 1.0 and will define the share of the dataset used for testing.

If you provide an int, then it will represent the total number of the training samples. The default value is None.

- **test_size** is the number that defines the size of the test set. It's very similar to train_size. You should provide either train_size or test_size.

If neither is given, then the default share of the dataset that will be used for testing is 0.25, or 25 percent.

- **random_state** is the object that controls randomization during splitting. It can be either an int or an instance of RandomState.

The default value is None.

- **shuffle** is the Boolean object (True by default) that determines whether to shuffle the dataset before applying the split.

Program to demonstrate splitting of single dimensional array using train_test_split

```
import numpy as np
from sklearn.model_selection import train_test_split

x=np.arange(1,25).reshape(12,2)
y=np.array([0,1,1,0,1,0,0,1,1,0,1,0])

print(x)
print(y)

x_train,x_test,y_train,y_test=train_test_split(x,y)
print("x_train==",x_train)
print("y_train==",y_train)

print("x_test==",x_test)
print("y_test==",y_test)
```

Given two sequences, like x and y here, `train_test_split()` performs the split and returns four sequences (in this case NumPy arrays) in this order:

1. `x_train`: The training part of the first sequence (x)
2. `x_test`: The test part of the first sequence (x)
3. `y_train`: The training part of the second sequence (y)
4. `y_test`: The test part of the second sequence (y)

```
>>> x_train, x_test, y_train, y_test = train_test_split(
...     x, y, test_size=4, random_state=4
... )
>>> x_train
array([[17, 18],
       [ 5,  6],
       [23, 24],
       [ 1,  2],
       [ 3,  4],
       [11, 12],
       [15, 16],
       [21, 22]])
>>> x_test
array([[ 7,  8],
       [ 9, 10],
       [13, 14],
       [19, 20]])
>>> y_train
array([1, 1, 0, 0, 1, 0, 1, 1])
>>> y_test
array([0, 1, 0, 0])
```

Earlier, you had a training set with nine items and test set with three items. Now because of argument `test_size=4`, the training set has eight items and the test set has four items. We will get the same result with `test_size=0.33` because 33 percent of twelve is approximately four.

x		y			
1	2	0	17	18	1
3	4	1	5	6	1
5	6	1	23	24	0
7	8	0	1	2	0
9	10	1	3	4	1
11	12	0	11	12	0
13	14	0	15	16	1
15	16	1	21	22	1
17	18	1	7	8	0
19	20	0	9	10	1
21	22	1	13	14	0
23	24	0	19	20	0

Training set

Test set

Linear Regression

Supervised learning methods: It contains past data with labels which are then used for building the model.

- **Regression:** The output variable to be predicted is *continuous* in nature, e.g. scores of a student, diamond prices, etc.
- **Classification:** The output variable to be predicted is *categorical* in nature, e.g. classifying incoming emails as spam or ham, Yes or No, True or False, 0 or 1.

What is linear regression?

Regression analysis is a statistical method that helps us to understand the relationship between dependent and one or more independent variables.

Dependent Variable

This is the Main Factor that we are trying to predict.

Independent Variable

These are the variables that have a relationship with the dependent variable.

- Linear Regression is a machine learning algorithm based on supervised learning. It performs a regression task. Regression models a target prediction value based on independent variables. It is mostly used for finding out the relationship between variables and forecasting.
- Linear Regression (LR) means simply finding the best fitting line that explains the variability between the dependent and independent features very well.
- It describes the linear relationship between independent and dependent features, and in linear regression, the algorithm predicts the continuous features (e.g. Salary, Price), rather than deal with the categorical features (e.g. cat, dog).
- Linear regression is one of the most commonly used techniques in statistics. It is used to quantify the relationship between one or more predictor variables and a response variable.

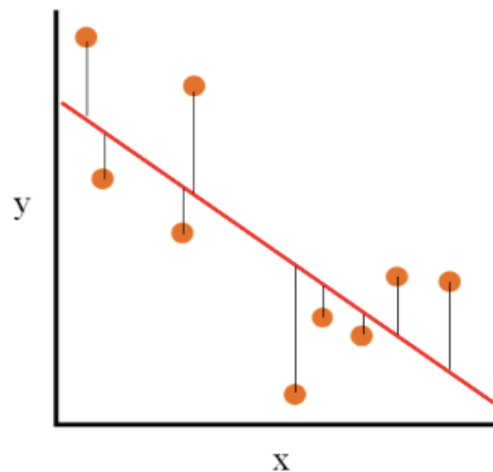
There are many types of regression analysis,

1. Simple Linear Regression.

Simple linear regression is a regression model that estimates the relationship between one independent variable and one dependent variable using a straight line. Both variables should be quantitative.

In the figure above, X (input) is the work experience and Y (output) is the salary of a person.

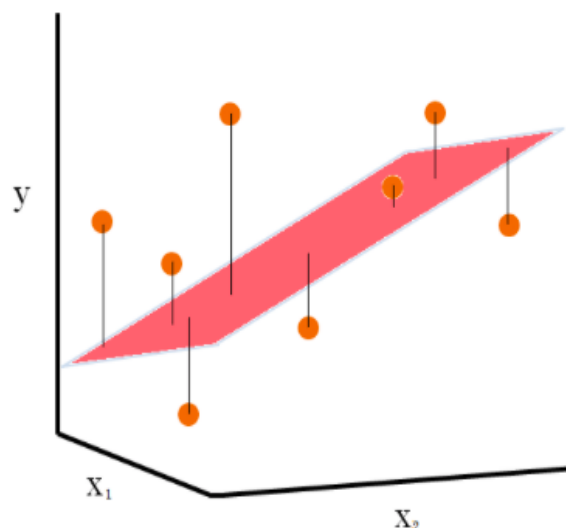
The regression line is the best fit line for our model.



2. Multiple Linear Regression.

Multiple linear regression (MLR), also known simply as multiple regression, is a statistical technique that uses several explanatory variables to predict the outcome of a response variable.

Multiple regression works by considering the values of the available multiple independent variables and predicting the value of one dependent variable. Example: A researcher decides to study students' performance from a school over a period



3. *polynomial Regression.*

in Polynomial Regression is a form of linear regression in which the relationship between the independent variable x and dependent variable y is modelled as an n th degree polynomial. Polynomial regression fits a nonlinear relationship between the value of x and the corresponding conditional mean of y , denoted $E(y|x)$.

Polynomial regression is one of the machine learning algorithms used for **making predictions**. For example, it is widely applied to predict the spread rate of COVID-19 and other infectious diseases.

Regularization in ML

- Regularization refers to **techniques that are used to calibrate machine learning models in order to minimize the adjusted loss function and prevent overfitting or underfitting.**
- Using Regularization, we can fit our machine learning model appropriately on a given test set and hence reduce the errors in it.

Real life applications of linear regression

- Linear regressions can be used in business to **evaluate trends and make estimates or forecasts.**
- For example, if a company's sales have increased steadily every month for the past few years, by conducting a linear analysis on the sales data with monthly sales, the company could forecast sales in future months.
- Medical researchers often use linear regression **to understand the relationship between drug dosage and blood pressure of patients.** For example, researchers might administer various dosages of a certain drug to patients and observe how their blood pressure responds.
- Agricultural scientists often use linear regression to measure the effect of fertilizer and water on crop yields.
- For example, scientists might use different amounts of fertilizer and water on different fields and see how it affects crop yield. They might fit a multiple linear regression model using fertilizer and water as the predictor variables and crop yield as the response variable
- Data scientists for professional sports teams often use linear regression to measure the effect that different training regimens have on player performance.
- For example, data scientists in the NBA might analyze how different amounts of weekly yoga sessions and weightlifting sessions affect the number of points a player scores. They might fit

a multiple linear regression model using yoga sessions and weightlifting sessions as the predictor variables and total points scored as the response variable.

Model Evaluation & testing

There are 3 main metrics for model evaluation in regression:

1. *R Square/Adjusted R Square*
2. *Mean Square Error(MSE)/Root Mean Square Error(RMSE)*
3. *Mean Absolute Error(MAE)*

R Square measures how much variability in dependent variable can be explained by the model. It is the square of the Correlation Coefficient(R) and that is why it is called R Square.

$$R^2 = 1 - \frac{SS_{Regression}}{SS_{Total}} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

- R Square is calculated by the sum of squared of prediction error divided by the total sum of the square which replaces the calculated prediction with mean. R Square value is between 0 to 1 and a bigger value indicates a better fit between prediction and actual value.
- R Square is a good measure to determine how well the model fits the dependent variables. **However, it does not take into consideration of overfitting problem.** If your regression model has many independent variables, because the model is too complicated, it may fit very well to the training data but performs badly for testing data.

Mean Square Error (MSE)/Root Mean Square Error (RMSE)

While R Square is a relative measure of how well the model fits dependent variables, Mean Square Error is an absolute measure of the goodness for the fit.

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

- MSE is calculated by the sum of square of prediction error which is real output minus predicted output and then divide by the number of data points. It gives you an absolute number on how much your predicted results deviate from the actual number. You cannot interpret many insights from one single result but it gives you a real number to compare against other model results and help you select the best regression model.
- Root Mean Square Error (RMSE) is the square root of MSE. It is used more commonly than MSE because firstly sometimes MSE value can be too big to compare easily. Secondly, MSE is calculated by the square of error, and thus square root brings it back to the same level of prediction error and makes it easier for interpretation.

```
from sklearn.metrics import mean_squared_error
import math
print(mean_squared_error(Y_test, Y_predicted))
print(math.sqrt(mean_squared_error(Y_test, Y_predicted)))
# MSE: 2017904593.23
# RMSE: 44921.092965684235
```

Mean Absolute Error (MAE)

- Mean Absolute Error (MAE) is like Mean Square Error (MSE). However, instead of the sum of square of error in MSE, MAE is taking the sum of the absolute value of error.

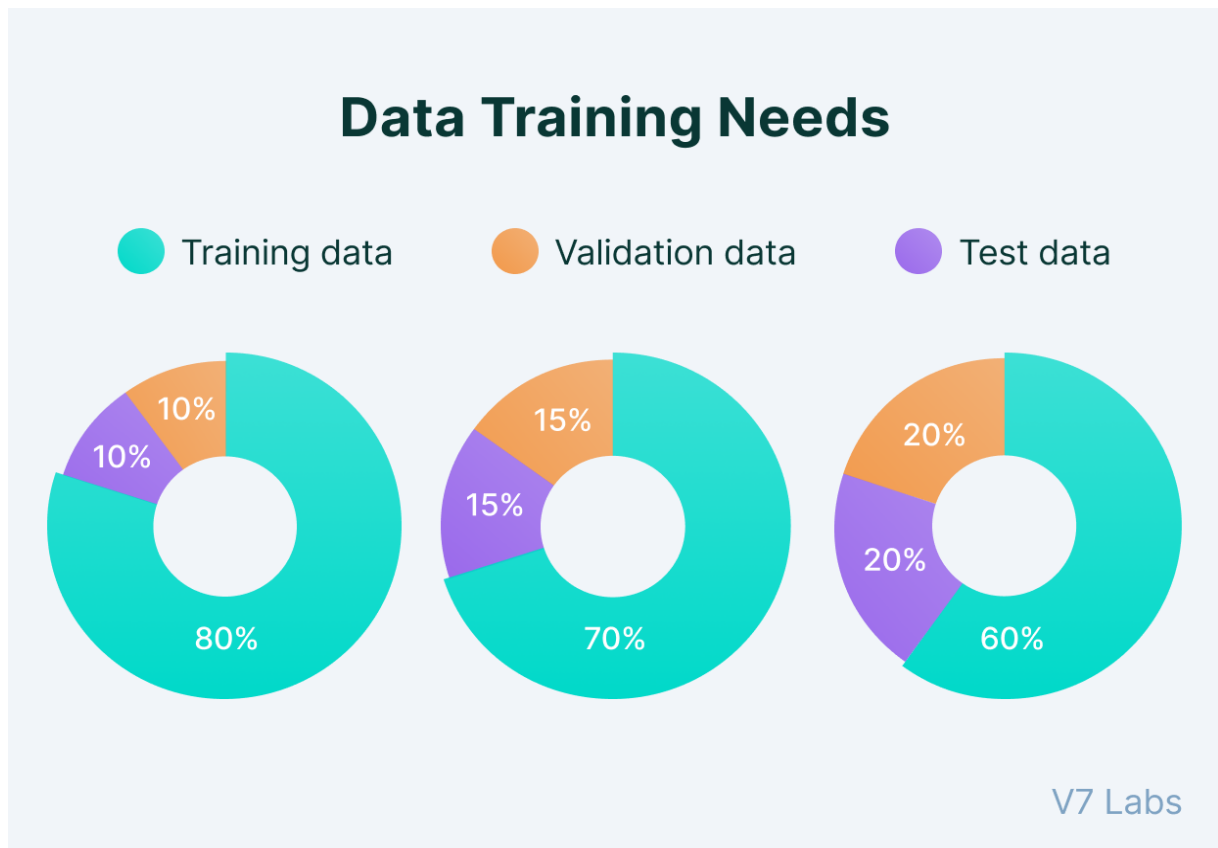
$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

- Compare to MSE or RMSE, MAE is a more direct representation of sum of error terms. **MSE gives larger penalization to big prediction error by square it while MAE treats all errors the same.**

```
from sklearn.metrics import mean_absolute_error
print(mean_absolute_error(Y_test, Y_predicted))
#MAE: 26745.1109986
```

What is model validation?

- ✓ Model validation is the process by which we ensure that our models can perform acceptable in “the real world.”
- ✓ model validation allows us to predict how our model will perform on datasets not used in the training (model validation is a big part of why preventing data leakage is so important).
- ✓ Model validation is important because we don’t actually care how well the model predicts data, we trained it on.
- ✓ We already know the target values for the data we used to train a model, and as such it is much more important to consider how robust and capable a model is when tasked to model new datasets of the same distribution and characteristics, but with different individual values from our training set.
- ✓ The first form of model validation introduced is usually what is known as holdout validation, often considered to be the simplest form of cross validation and thus the easiest to implement.



What is cross Validation?

- ▶ Cross-validation is a technique for validating the model efficiency by training it on the subset of input data and testing on previously unseen subset of the input data.
- ▶ Cross-validation is a technique in which we train our model using the subset of the data-set and then evaluate using the complementary subset of the data-set.

- ▶ There are many types of Cross-Validation techniques, and in this post I will talk about three of them
 - **Holdout,**
 - **K-Fold** and
 - **Leave-One-Out.**

Hold-out validation

- ▶ The most famous type of Cross-Validation technique is the Holdout.
- ▶ This technique consists in separating the whole dataset into two groups, without overlap: training and testing sets.
- ▶ This separation can be made shuffling the data or maintaining its sorting, depends on the project.
- ▶ It is common to see a 70/30 split in projects and studies, with 70% of the data being used to train the model and the remaining 30% being used to test and evaluate it.
- ▶ However, this ratio is not a rule and it may vary depending on the specificity of the project

```
# Dataset used
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

# Loading the data
data_wdbc = load_breast_cancer()

# Creating the dataframe with the features
X = pd.DataFrame(data = data_wdbc.data, columns = data_wdbc.feature_names)

# Defining the target
y = data_wdbc.target

print('Shape of the data:', X.shape)
# Output:
# Shape of the data: (569, 30)

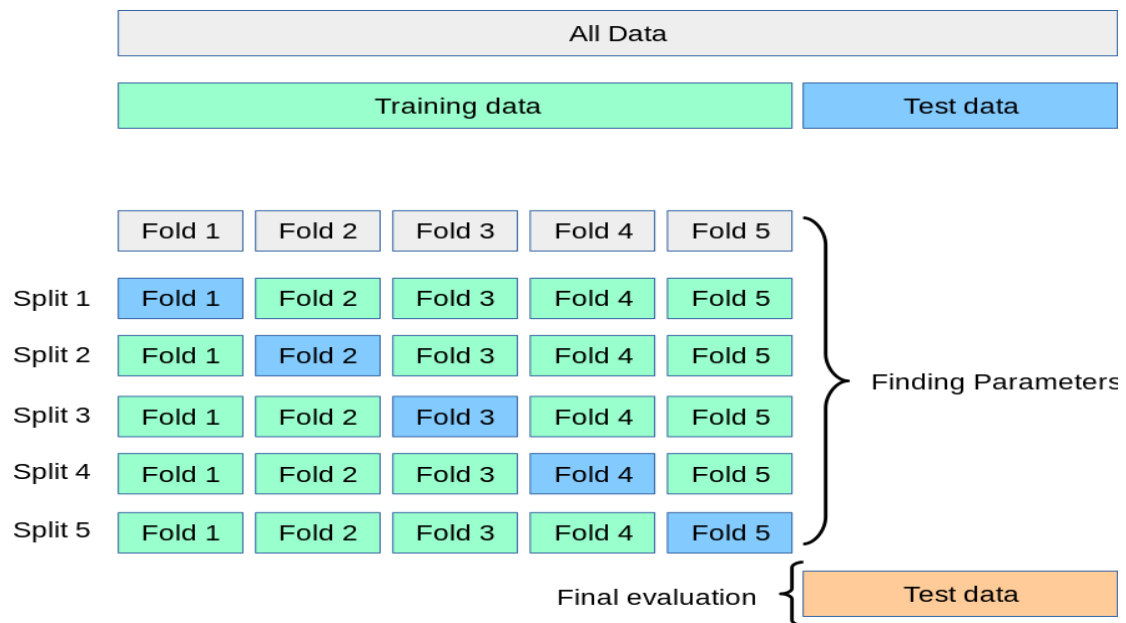
# Splitting the data with 30% being defined as test data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.3)

print('Shape of the Training set:', X_train.shape)
print('Shape of the Test set:', X_test.shape)
# Output:
# Shape of the Training set: (398, 30)
# Shape of the Test set: (171, 30)
```

K-Fold Validation

- ▶ Before separating the data into training and testing sets, the K-Fold Cross-Validation separates the whole data into K separated subsets with approximate size. Only then, each subset is divided into training and testing sets.
- ▶ Each subset is used to train and test the model.

- In practice, this technique generates K different models with K different results. The result of the K-Fold Cross-Validation is the average of the individual metrics of each subset.



```
from sklearn.datasets import load_iris
from sklearn.model_selection import cross_val_score, KFold
from sklearn.linear_model import LogisticRegression
iris=load_iris()
X=iris.data
Y=iris.target
logreg=LogisticRegression()
kf=KFold(n_splits=5)
score=cross_val_score(logreg,X,Y,cv=kf)
print("Cross Validation Scores are {}".format(score))
print("Average Cross Validation score :{}".format(score.mean()))
```

```
Cross Validation Scores are [1.          1.          0.86666667 0.93333333 0.83333333]
Average Cross Validation score :0.9266666666666665
```

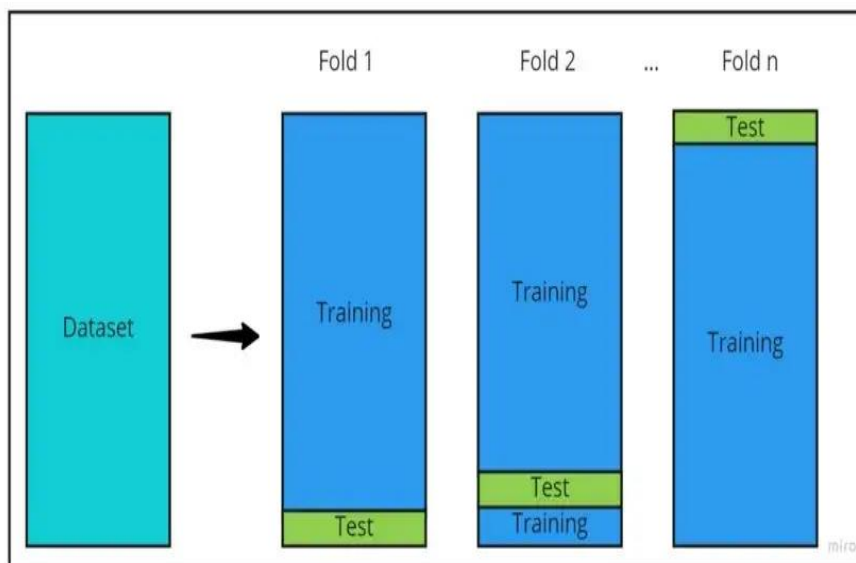
It is important to notice that since the K-Fold divides the original data into smaller subsets, the size of the dataset and the K number of subsets must be taken into account.

If the dataset is small or the number of K is too big, the resulting subsets may become very small.

This may result in just a few data to be used to train the models, resulting in a poor performance since the algorithm could not understand and learn the patterns in the data due to lack of information.

Leave-one-out cross validation

- ▶ The Leave-One-Out Cross-Validation consists in creating multiple training and test sets, where the test set contains only one sample of the original data and the training set consists in all the other samples of the original data. This process repeats for all the samples in the original dataset.
- ▶ This type of validation usually is very consuming because if the data used contains n samples, the algorithm will have to train (using $n-1$ samples) and evaluate the model n times.
- ▶ On the positive side, this technique, of all seen in this post, is the one in which the models used have the largest number of samples used for training, and this may result in better models developed. Also, there is no need to shuffle the data, since all possible combinations of train/test sets will be generated.



```
from sklearn.datasets import load_iris  
from sklearn.ensemble import RandomForestClassifier  
from sklearn.model_selection import LeaveOneOut,cross_val_score  
  
iris=load_iris()  
X=iris.data  
Y=iris.target  
  
loo=LeaveOneOut()  
  
tree=RandomForestClassifier(n_estimators=10,max_depth=5,n_jobs=-1)  
score=cross_val_score(tree,X,Y,cv=loo)  
print("Cross Validation Scores are {}".format(score))  
print("Average Cross Validation score :{}".format(score.mean()))
```

Cross Validation Scores are [1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.
1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.
1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.
0. 1. 1. 1. 1. 0. 1. 1. 1. 1. 1. 0. 1. 1. 1. 1. 1. 1. 1. 1. 1.
1. 1. 1. 1. 1. 1. 1. 1. 1. 0. 1. 1. 1. 1. 1. 1. 1. 1. 1. 0.

Program to demonstrate Analysis of CIE and SEE using linear regression model.

```
import pandas as pd
import numpy as np

df=pd.read_csv("C:/Users/Shilpa/Desktop/dataset/marks1.csv")
df.info ()

x = df['CIE'].values.reshape(-1,1)
y = df['SEE'].values.reshape(-1,1)

from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split (x, y,random_state =0)
from sklearn.linear_model import LinearRegression

lm = LinearRegression()
lm.fit(x_train, y_train)
y_pred = lm.predict(x_test)

from sklearn.metrics import mean_squared_error,mean_absolute_error,r2_score
g=y_test.reshape(21,)
h=y_pred.reshape(21,)

mydict={"Actual": g,"Pred":h}
com=pd.DataFrame(mydict)
com.sample(10)

def evaluationmatrices(Actual,Pred):
    MAE=mean_absolute_error(Actual,Pred)
    MSE=mean_squared_error(Actual,Pred)
    RMSE=np.sqrt(mean_squared_error(Actual,Pred))
    SCORE=r2_score (Actual,Pred)
    return print ("r2 score:",SCORE,"\n","MAE", MAE,"\n","mse",MSE,"\n","RMSE",RMSE)

evaluationmatrices(g,h)
```

Program to demonstrate House price prediction using multi linear regression.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df=pd.read_csv("C:/Users/Shilpa/Desktop/Housing (1).csv")
df.head(10)

df.info()
df.describe()

status=pd.get_dummies(df['furnishingstatus'])
col=['mainroad', 'guestroom', 'basement', 'hotwaterheating', 'airconditioning', 'prefarea']

def binary_map(x):
    return x.map({'yes': 1, "no": 0})
df[col] = df[col]. apply(binary_map)

status = pd.get_dummies(df['furnishingstatus'], drop_first = True)
df= pd.concat([df, status], axis = 1)
df.head(10)

df.drop(['furnishingstatus'], axis = 1, inplace = True)

x= df.iloc[:,1:]
y=df.iloc[:,0]

from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size = 0.3, random_state = 100)

from sklearn.linear_model import LinearRegression
lm = LinearRegression()
lm.fit(x_train,y_train)
y_test=np.array(y_test)

y_test=y_test.reshape(-1,1)
y_pred=lm.predict(x_test)

from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

g=y_test.reshape(164,)

my_dict = {"Actual": g, "Pred" : y_pred}
compare = pd.DataFrame(my_dict)
compare.sample(10)
```

```
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
def evaluation_metrics(actual, pred):
    MAE = mean_absolute_error(actual, pred)
    MSE = mean_squared_error(actual, pred)
    RMSE = np.sqrt(mean_squared_error(actual, pred))
    SCORE = r2_score(actual, pred)
    return print ("r2 score:", SCORE, "\n", "MAE:" , MAE, "\n", "mse: ", MSE, "\n", "rmse:" ,
RMSE)
evaluation_metrics(g, y_pred)

from yellowbrick.regressor import PredictionError
visualizer = PredictionError(lm)
visualizer.fit(x_train, y_train)
visualizer.score(x_test,g)
visualizer.show()
```